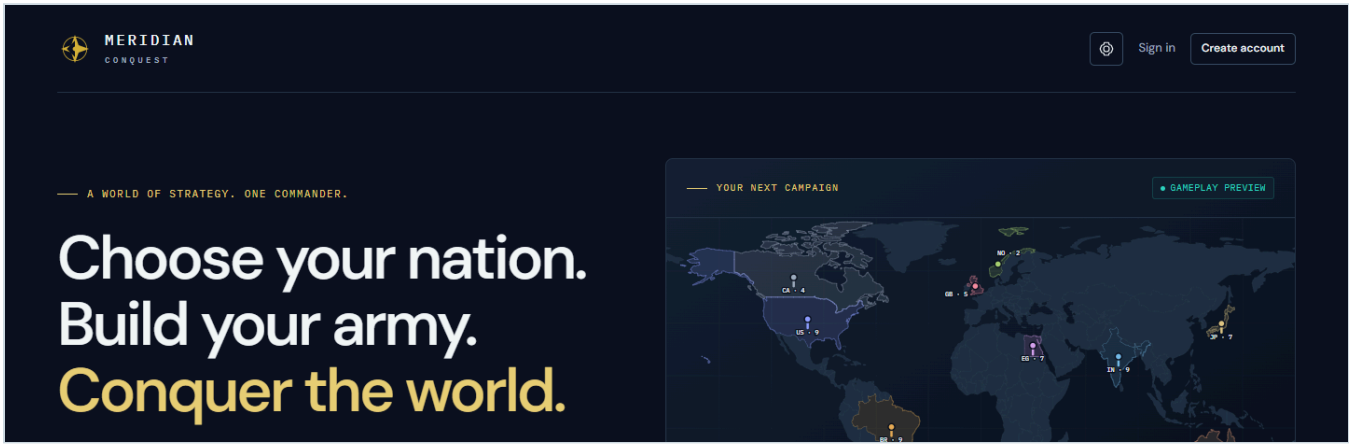
Meridian Conquest

Project documentation · INF4001N entrance assignment

A turn-based strategy game in which ten real countries compete until one commander controls the entire board.

Live application: https://meridian-conquest.vercel.app
Repository: https://github.com/Muhammed112/Meridian_Conquest_UI



Overview

Meridian Conquest is a turn-based geopolitics strategy game. A player selects a country, receives a population-based army and faces nine strategic NPC commanders. Each turn combines reinforcement decisions, optional transfers and one attack target. The victory condition is ownership of all ten countries.

Extra features include multiplayer rooms, temporary NPC alliances, an interactive adviser tutorial, replay with AI key-event summaries, sound and animation, and public live viewing. Gemini narrates recorded facts without deciding combat outcomes.

Contents

1. Overview and project information
2. System architecture and rationale
3. Key features and game rules
4. Getting started and environment configuration
5. macOS ZIP and GitHub startup methods
6. System components and database design
7. User roles and workflows
8. Replay and AI summaries
9. Verification, troubleshooting and limitations
10. Project structure, documentation and acknowledgements

System architecture

Browser game logic with managed Firebase services

The application uses HTML, CSS and native JavaScript ES modules. Vercel serves the maintained dist source directly. There is no frontend compilation step or deployed custom game server.

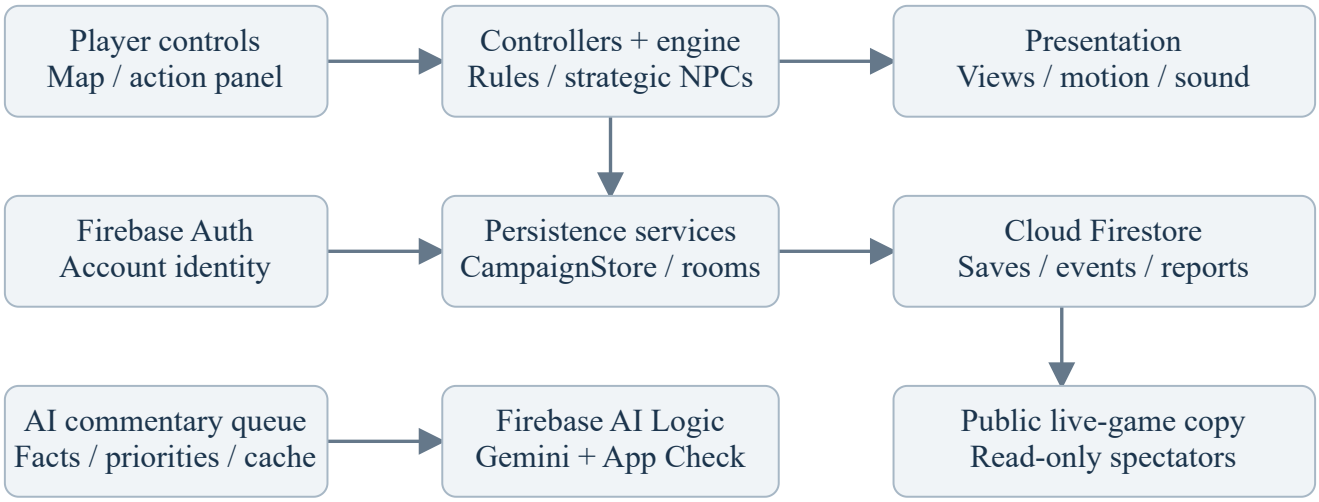


Figure 1. Main responsibilities and external services. Game state and AI narration follow separate paths.

Architecture rationale

Decision	Reason
Separate game engine	Rules can be tested without a browser, database or AI model.
Shared views and controllers	Single-player and multiplayer reuse controls while resolving the current viewer’s permissions.
Cloud Firestore	Document saves, append-only events and live listeners suit independent campaigns and shared rooms.
Firebase Authentication	Credentials remain with the authentication service; application passwords are not stored in Firestore.
Independent commentary queue	Slow or unavailable model requests do not block game turns.
Bundled assets and snapshots	The map, flags and country data remain available when a source-data refresh fails.

Key features and game rules

Required gameplay and extensions

Core functionality

- Ten distinct countries: the player chooses one and nine rivals are drawn randomly. Multiple campaigns can be saved independently.
- Starting armies follow the brief’s population bands. The catalogue contains 233 selectable countries and territories.
- All commanders roll for the first turn. Tied leaders reroll; the final turn-order preview appears before play starts.
- NPCs plan reinforcements, transfers, attacks and retreats. Their reasons remain visible during fast-forward.
- Paired dice, casualties, captures, elimination and victory are displayed during play.

Additional features

- Four difficulty levels: Easy, Medium, Hard and Extreme.
- Six-character multiplayer room codes, country reservations and bot-filled seats.
- Three-round NPC alliances with trust, expiry and betrayal.
- Captain Mara’s guided tutorial with typed explanations and control pointers.
- Live spectator links, player-left notices and completed-game recordings.
- On-demand AI replay summaries with specific key events and View moment links.
- Replay, flags, distant-attack planes, music and reduced-motion settings.

Event and narration flow

1. A controller receives a player action or an NPC plan and calls the game engine.
2. The engine checks legal moves, changes the board and records factual events.
3. Views display the board; presentation timing hides casualties until the dice settle.
4. The persistence service saves the board and immutable facts under the permitted account or room.
5. Eligible events enter the priority queue. Gemini receives compact event-time facts through Firebase AI Logic.
6. Accepted narration is stored separately and shown in the AI commentary tab. An explicit public copy can deliver available reports to spectators.

Game rules and player permissions

Combat rules and access for each type of user

Rule	Implementation
Starting soldiers	1–9 soldiers using population upper bounds of 5, 10, 25, 50, 75, 100, 125 and 150 million; larger populations receive 9.
Reinforcements	From round two: 2 soldiers per owned country, allocated among the commander’s holdings.
Attacks	One enemy target per turn. Any location is reachable. The attacking country needs at least 2 soldiers.
Dice	Each side rolls min(attacker army − 1, defender army) dice. Compare corresponding pairs; lower dice lose a soldier, while ties cause no loss.
Capture	Reward = max(2, floor(original country soldier value / 2)). One earned soldier garrisons the captured country.
Transfers	Only between the same commander’s countries, outside active combat; retain at least one soldier at the origin.
End conditions	Stopping an attack ends the turn. A commander with no countries is eliminated. Ownership of all ten wins.

> Example: an attacker with 10 soldiers against a defender with 2 rolls two pairs. Results 1 vs 2 and 4 vs 3 remove one soldier from each country, leaving 9 vs 1.

Role	Permitted interaction
Signed-in player	Create and resume private campaigns, play turns, manage a nickname and explicitly share a campaign.
Multiplayer host	Create/start a room, prepare opening rolls, control bot turns and generate shared room narration.
Multiplayer friend	Join by code, choose a distinct country, roll individually and act only during their own turn.
Public spectator	Read an explicitly shared live or recorded board without sign-in; cannot play or negotiate.
Project operator	Configure Firebase/hosting, run deliberate integration tools and publish tested releases; not an in-game administrator role.

Getting started and configuration

Local installation and hosted operation

Run the supplied ZIP — no GitHub access needed

Install **Node.js 24 with npm**. Use a modern browser and internet access to Google/Firebase services, even on localhost. Java 21+ is needed only for emulator tests.

1. Extract the ZIP completely. Open a terminal in the extracted folder containing package.json and package-lock.json, not inside dist.
2. Run the two commands below, in order:

```
npm ci
npm run dev
```

Keep the terminal open. When it prints Meridian Conquest: http://localhost:8081, open **http://localhost:8081** in Chrome or another modern browser. Create a player account or use the application test credentials supplied separately. Press **Ctrl+C** in the terminal to stop.

Do not start by double-clicking dist/index.html. Ordinary gameplay, sign-in and cloud saves need no build, .env.local, private cache folders or administrator CLI login. The required public configuration is included.

To use port 8082 in PowerShell:

```
$env:MERIDIAN_PORT = '8082'
npm run dev
```

Included configuration

Location	Purpose
dist/firebase-config.js	Public settings for meridian-conquest-m112; not administrator credentials.
dist/ai-config.js	Public Gemini model and reCAPTCHA site settings; no personal Gemini API key.
dist/ai-local-config.js	Empty public stub; the local server can supply a private App Check attestation in memory.

macOS and GitHub startup

Run the supplied ZIP or clone the repository

Mac ZIP method: Python 3

The browser application runs on macOS. With **Python 3** and VS Code installed, the supplied ZIP can be served without installing Node dependencies:

1. Extract the ZIP completely and open **Meridian_Conquest_UI** in VS Code. The opened folder should contain dist.
2. Select **Terminal → New Terminal**. Run the command below from that project folder:

```
python3 -m http.server 8080 --directory dist
```

Keep the terminal open and visit http://localhost:8080 in a modern browser. Create a player account or sign in with an existing player email and password. Press **Ctrl+C** in the terminal to stop the server.

> Python serves static files only. Gameplay, sign-in and cloud saves use the included public configuration and require internet access. This server does not provide local App Check attestation or background population checks. For AI commentary and summaries, use the deployed application, or the Node development server with the operator App Check setup in Getting started and configuration.

GitHub method: macOS or Windows

This method requires **Git** and **Node.js 24 with npm**. The repository is public, so no GitHub account or access grant is needed to clone it. The supplied ZIP is an alternative.

In **Terminal → New Terminal**, run:

```
git clone https://github.com/Muhammed112/Meridian_Conquest_UI.git
cd Meridian_Conquest_UI
npm ci
npm run dev
```

Open the address printed by the development server, normally http://localhost:8081. The same Node commands also work from an extracted ZIP on macOS or Windows. Local AI retains the App Check requirements in Getting started and configuration.

System components and AI setup

Responsibilities, encapsulation and persistence

Component	Responsibility and representative API
GameEngine dist/engine.js	Applies rules and records facts: reinforce(), transfer(), startAttack(), roll(), stopAttack(), endTurn() and proposeAlliance().
BattleEngine / NpcStrategy	BattleEngine.resolve() compares simultaneous dice. NPC plans evaluate force, rewards, threats, reserves and alliances without changing dice.
CampaignStore	list(), open() and save() separate board summaries from paginated event histories and detect competing saves.
Room service	Country choices, opening dice, host-controlled bot turns, room revisions and transactions.
CommentaryQueue	enqueue() and drain() manage event priority, deduplication, cached reports and asynchronous generation.
Replay summaries dist/game-summary.js	Ranks bounded facts across the available history; a separate Gemini prompt and modal provide concrete highlights without writing game state.
UI and presentation	Shared render helpers/controllers, map interactions, typed tutorial dialogue, MotionDirector and SoundDirector.

AI commentary and summaries: hosted or local

For AI features without operator setup, open https://meridian-conquest.vercel.app and sign in as a player. Local AI additionally needs an App Check debug attestation registered by an operator with permission on the Firebase project. A player account or read-only database access does not grant that permission.

Without this attestation, local AI may show a setup warning; the server, gameplay and saves still work. Operators can run npx firebase login and restart the server; see docs/FIREBASE.md.

The GitHub repository is public; cloning does not require a GitHub account or an access grant. The supplied ZIP is an alternative. Vercel serves dist; Firebase rules are published separately. The optional functions implementation remains undeployed on the Spark plan.

Database and validation

Persistence, access controls and known limitations

Cloud Firestore collections

Path	Purpose / access
users/{uid}/games/{gameId}	Private campaign board, listing fields and save revision; signed-in owner access.
…/events/{eventId}	Immutable factual event, sequence and predecessor link.
…/aiReportsV2/{eventId}	Separate narration with event reference, model, version and server timestamp.
rooms/{code}	Members, country reservations, opening dice and shared board; controlled member updates.
rooms/{code}/events rooms/{code}/aiReportsV2	Room event history and host-generated commentary.
liveGames/{gameId} …/presence/{sessionId}	Explicit public projection and owner-written presence heartbeats.
publicData/populations	Validated public population reference; restricted updater writes.

Save and validation design

Campaign headers load independently from history. Selected histories load in pages of up to 250 events. New events are staged immutably; a transaction commits the new revision and history head. Readers follow the committed chain, excluding unfinished or competing branches. Legacy records remain readable.

Firebase Authentication manages passwords. Firestore rules enforce ownership, membership, turn permissions and bounded transitions. Generated text is excluded from immutable combat facts. Public projections select gameplay fields and exclude account email addresses and credentials.

Known limitations

- Dice and full combat execute in the browser. Rules enforce substantial storage/turn constraints, but there is no authoritative competitive anti-cheat server.
- Gemini response time and quotas vary. Event priorities skip stale narration; factual reports and gameplay continue independently.
- Population refresh failures retain the last validated snapshot. Existing games keep their saved populations and rewards.

User roles and workflows

Players, room members, visitors and operators

Single-player and tutorial

1. Create an account or sign in and choose a nickname. New players without campaigns are offered guided practice.
2. Select New campaign, a difficulty and a searchable starting country; inspect the nine random rivals.
3. Roll, resolve tied leaders and inspect Turn order. Select Start campaign.
4. From round two, allocate reinforcements. Transfer, attack one target, negotiate an NPC pact or skip as permitted.
5. Read factual reports or AI narration. Resume the campaign later or inspect chronological replay.

Multiplayer and alliances

The host shares a six-character invitation code. Friends sign in and reserve distinct countries. The host prepares opening dice; empty seats become bots. Each human rolls, tied leaders reroll and the host starts after the order is resolved. The host keeps the room open for NPC playback.

Select **Alliance** during a clear planning phase to offer an NPC a three-round pact. One ally is allowed at a time. Armies and turns remain separate. Attacking an ally breaks the pact and reduces trust. Expiry, elimination or only the allies remaining ends the agreement. Human-to-human proposals are outside this version.

Campaign cards, sharing and viewing

Active cards provide **Resume campaign**; share ongoing games using **Share live link** in the in-game reports header. Finished cards provide **View results**, **Replay** and **Share gameplay**. An existing public share retains its stop control. Live links show updates and player-left notices; abrupt exits take about 80 seconds to detect. Recorded links contain up to 60 recent events and saved commentary, and cannot be revoked.

Pin reports bounds the feed beneath a proportionally scaled map; both scroll together and messages scroll internally. **Stop sharing** removes the live public copy, preserving the private campaign.

Replay and AI summaries

Specific key events with links to their recorded evidence

Using the summary

In **Battle replay**, select **AI summary** beside the playback controls. The replay pauses while the summary window is open. Each highlighted event includes its round and a **View moment** button that closes the window and selects the exact source event.

State	What the viewer sees
Generating	A loading message without the event list. Gameplay facts remain unchanged.
Summary ready	Concrete key events first, followed by a short AI recap. Gemini selects up to eight highlights, with at least five when available.
Generation failed	Recorded key moments and an explicit Retry AI summary button. There is no automatic retry loop.
Signed-out viewer	Recorded facts without making a Gemini request. Signing in enables AI generation.

What counts as a key event

- Captures: named commander, captured country, known previous owner, territory changes and recorded rewards.
- Eliminations and victory: the defeated commander and final country where available, or the commander who controls all ten countries.
- Major battles: exact countries and participants, with recorded casualty totals. Missing earlier phases are explicitly treated as unknown.
- Alliance outcomes: accepted or broken pacts, expiry and the commanders involved.

Facts, access and request limits

The summary ranks the available history into at most twelve factual candidates before making one bounded Gemini request. It uses a separate prompt from live commentary and works independently of that setting. Country counts, rather than troop totals, determine territorial leads. Generated wording cannot replace the visible recorded facts.

Reopening the same record on the current page reuses its result. The bounded cache is account-specific and kept only in memory; refresh or account changes clear it. Summaries never change saved gameplay or create Firestore records. They use the existing Firebase AI Logic and App Check setup and remain subject to provider quotas.

> A shared replay contains only its available recent history, usually up to 60 events. The summary cannot reconstruct earlier battles or claim to describe an unrecorded part of the match.

Verification and troubleshooting

Reproducible checks and practical boundaries

I shared the deployed game link with other people for user testing and asked them to identify bugs and suggest UI improvements. Their feedback helped me refine gameplay pacing, tutorial guidance, country-label visibility and the report layout.

Command	Purpose
npm run check	Formatting, ESLint and unit tests.
npm run browsers:install npm run test:browser	Install Chromium, then run isolated browser gameplay/UI fixtures. Installed Chrome is also supported.
npm run test:rules	Firestore permission tests against demo-meridian on port 8180; requires Java 21+.
npm run test:ai	Offline commentary-validation cases, without a model request.
npm run commentary:preview	Deliberate real-Gemini evaluation; needs operator/local setup and uses provider quota.

The release passed the GitHub application and Firestore checks, including 306 unit tests, 18 permission tests and the offline browser suites. Published-file verification matched 306 non-audio assets and checked eight private paths. Summary tests cover concrete facts, loading, cache reuse, account isolation and source-event links without live model requests. Sharing tests cover delayed restoration and revocation retries. Passing checks cover their fixtures; they do not prove every possible interaction correct.

Troubleshooting

Symptom	Check
Old labels or layout	Refresh with Ctrl+F5 after an update.
Local port unavailable	Set MERIDIAN_PORT to an unused port and restart the server.
Local AI unavailable	Use npm run dev with operator App Check setup, or try the deployed site. Check the visible AI status and provider quota.
Campaigns missing	Confirm the signed-in account; private saves are account-specific.
Friend cannot open localhost link	Share from the deployed site. Loopback addresses refer to the viewer’s own computer.
Multiplayer bots stop	Keep the host’s room open and connected.

Project structure and references

Source organisation, supporting guides and credits

```
Meridian_Conquest_UI/
  dist/                    Browser source and bundled assets
    controllers/           Actions, campaign loading, NPC playback
    ui/                    Shared screens, map, reports and tutorial
  scripts/                 Dev server, data tools, browser checks
  tests/                   Unit and Firestore permission tests
  docs/                    Architecture, setup and user guidance
  functions/               Optional undeployed commentary backend
  firestore.rules          Firestore access and transition rules
  firestore.indexes.json   Index settings
  firebase.json            Firebase deployment/emulator settings
  vercel.json              Static hosting configuration
  package.json             Dependencies and project commands
  README.md                Installation and usage guide
  CONTRIBUTING.md          Development practices
```

Additional documentation

File	Purpose
README.md / CONTRIBUTING.md	Installation, usage, support and development checks.
docs/ARCHITECTURE.md docs/DATABASE.md	Responsibilities, class boundaries, schema, save compatibility and access.
docs/FIREBASE.md docs/VERCEL.md	Service setup, hostname registration and deployment procedures. Historical release sections are dated.
docs/TUTORIAL.md docs/DIPLOMACY.md	Instruction coverage and pact behaviour. Older guides may call the Alliance panel “Diplomacy”.
docs/email-templates/	Password-reset template and its setup guidance.

Author, support and contributions

Developed by **Muhammed (Muhammed112)** for the INF4001N entrance assignment. Report reproducible issues through the repository’s issue tracker. Contributions should preserve game rules and save compatibility, reuse shared logic and include appropriate checks.

Acknowledgements and licences

Population figures: Worldometers. Country metadata: mledoze/countries (ODbL 1.0). Map: world-atlas (ISC) and Natural Earth geometry. Flags: flag-icons (MIT). Music: Kevin MacLeod’s Decisions, Lost Frontier and Mechanolith (CC BY 4.0). Sound effects: Kenney CC0 samples and original Web Audio cues. The interface builds on the supplied Figma design.

Asset notices remain in dist/assets; music attribution is also visible in Sound settings. No separate project-wide open-source licence is included for the original application code.